Justin Jenkins

COIN 496 Senior Portfolio

February 28, 2010

Program Introductory Page

COIN 315 Data Structures

The two programs are an object-oriented programming type project that was created for a Data Structure class in spring 2009. The best way of running the programs is using Microsoft Visual Studio 2008 but it is also compatible running under Microsoft Visual Studio 2005. The process of running the programs can be followed by creating a new project and adding the existing programs files to the project. After setting up the programs, you can build and run the programs which will appear in the command prompt.

COIN 325 Java I

This program was created for a Java I class in spring 2008. Written under the Java programming language, the program can be best compiled under NetBeans IDE application. The process of compiling this program in NetBeans IDE is to create a new project and naming it as the exact class name, which is "Calculate2". After setting up the program, you can build the main project and run it from the output screen located within the application window. COIN 333

COIN 333 Applied Systems

This program was created for an Applied Systems class in spring 2009. Written under the Perl programming language, the program can be ran under Windows, Mac OS X, or Linux operating systems. The program was tested under the Windows XP operating system using the Notepad and ActivePerl 5.10.1 Build 1007. It was compiled from the command prompt using the command:
"perl (path)/HW9.pl"

```
//Student Name: Justin Jenkins
//Program Name: Index Multi List (Project 2)
//Creation Date: Spring 2009
//Last Modified Date: Spring 2009
//COIN Course: COIN 315 Data Structures
//Grade Received: 90
//Comments regarding design: Object-Oriented Programming Type Project.  Written in C++ language.


//const int DEFAULT_MAX_ITEMS=20;

class IndexedMultiList {
public:
//Constructors/destructors
    IndexedMultiList();//creates an empty list capable of storing
//DEFAULT_MAX_ITEMS items
    ~IndexedMultiList();    //destructor ONLY if needed

//accessors
    bool empty() const;//returns true iff (if and only if) list is empty
    int uniqueSize() const;//returns number of positions in the list in use
    int size() const;   //returns total number of items in list using counts
//If 2 A, 3 C, 1 D are inserted into an IndexedMultiList iml then
//iml.uniqueSize() would return 3 and iml.size() would return 6.
    int find(char val) const;
//returns index of first position with val in it or -1 if not found
    int countItem (char val) const;
//returns number of locations val is in the list (vacuously 0 if not //found)
    int totalItemCount (char val) const;
/*returns number of val in the list (vacuously 0 if not found)
Example: there are 3 A's in the first position, 6 B's in the second and 4 A's in the third position.
find('A') returns 0, find('B') returns 1 and find('C') returns -1
countItem('A') returns 2  (it is in the first and third location)
countItem('B') returns 1, countItem('C') returns 0
totalItemCount('A') returns 7, totalItemCount('B') returns 6,
totalItemCount('C') returns 0
*/

    int countPosition(int index) const;//returns number of items at position index (0 based) or 0 if index is invalid
    bool getAt(int index, char &out) const;
//if position index is in use, out is set to char at that position and true is returned, otherwise false is returned.
    void print() const;
//prints out the content of the list in order by position, see below

//mutators
    bool insert(int index, char val, int count);
/*insert count number of val into the list at position index
returns true iff val was able to fit in list and index was valid and count >=1
valid indices range from 0 up to unique size (inclusive).  So for an empty list, 0 is the only valid index.
insert should do nothing if index is invalid
If the list has 3 A's in the first position, then 0 and 1 are the only valid indices.
*/
    bool change(int index, int amount);
/*add amount to item in position index
returns true iff index was valid
change should do nothing if index is invalid
amount could be negative, if adding amount to the current count results in a count of <= 0 then position should be removed
For example, if there are 3 A's at index 0 and 2 B's at index 1 in the list iml then
iml.change(0, -4)
would remove all the A's and the 2 B's would be at index 0.
```

```
*/

    bool update(char val, int amount); //finds val in list and adds amount (which could be negative)
//return true iff found, works similarly to change but is based on the
//character to find instead of the position.  If val is in multiple //locations within the list, then only change the first location

    bool remove(char val);  //removes 1 instance of val from list, //returns true iff val was in list, if val is in multiple locations,
//then only remove from the first location

    int remove(int index);  //remove position at index, returns count //of items removed or -1 if index not in use

    int removeAll(char val); //removes all instances of val from list
//returns number of items removed, which could be 0, if val is in //multiple locations it should remove only the first location

private:
    struct Node
    {
        Node(char d, int c, int in, Node *n = 0) : data(d), count(c), index(in),next(n)
    {}
        char data; // The value itself (node value) to be placed in the list
        int count; // Stores how many values are in a spot
        int index; // Designated spots where a value can be placed in
        Node *next;
    };
    Node *head;
};
```

```cpp
#include <iostream>
#include "IndexedMultiList.h"
using namespace std;


//Constructors/destructors
IndexedMultiList::IndexedMultiList()//creates an empty list capable of storing
{
    head = NULL;
}

IndexedMultiList::~IndexedMultiList()   //destructor ONLY if needed
{
    Node * current = head;
    while (head != NULL)
    {
        current = head;
        head = head -> next;
        delete current;
    }
}

//ACCESSORS
//returns true iff (if and only if) list is empty
bool IndexedMultiList::empty() const
{
    return (head == NULL);
}

//returns number of positions in the list in use
int IndexedMultiList::uniqueSize() const
{
    int total = 0;
    Node * current = head;
    while (current != NULL)
    {
        total++;
        current = current->next;
    }
    return total;
}


//returns total number of items in list using counts
//If 2 A, 3 C, 1 D are inserted into an IndexedMultiList iml then
//iml.uniqueSize() would return 3 and iml.size() would return 6.
int IndexedMultiList::size() const
{
    int total = 0;
    Node * current = head;
    while (current != NULL)
    {
        total+= current ->count;
        current = current ->next;
    }
    return total;
}

//returns index of first position with val in it or -1 if not found
int IndexedMultiList::find(char val) const
{
    Node * current = head;
    while (current ->next != NULL && current -> data != val)
        current = current ->next;
    if (current ->data == val)
        return current->index;
```

```
    else
        return -1;

}

//returns number of locations val is in the list (vacuously 0 if not //found)
int IndexedMultiList::countItem (char val) const
{
    int total = 0;
    Node * current = head;
    while (current -> next != NULL)
    {
        if (current -> data == val)
            total+=1;
        current = current ->next;
    }
    return total;
}

/*returns number of val in the list (vacuously 0 if not found)
Example: there are 3 A's in the first position, 6 B's in the second and 4 A's in the
    third position.
find('A') returns 0, find('B') returns 1 and find('C') returns -1
countItem('A') returns 2  (it is in the first and third location)
countItem('B') returns 1, countItem('C') returns 0
totalItemCount('A') returns 7, totalItemCount('B') returns 6,
totalItemCount('C') returns 0
*/
int IndexedMultiList::totalItemCount (char val) const
{
    Node * current = head;
    int total=0;
    while (current -> next != NULL)
    {
        if(current->data == val)
            total+=current->count;
        current = current ->next;
    }
    return total;
}

//returns number of items at position index (0 based) or 0 if index is invalid
int IndexedMultiList::countPosition(int index) const
{
    Node * current = head;
    if (index < 0)
        return -1;
    else
    {
        while (current != NULL && current ->index != index)
            current = current ->next;
        if (current ->index == index)
            return current->count;
        else
            return 0;
    }
}
//if position index is in use, out is set to char at that position and
//true is returned, otherwise false is returned.
bool IndexedMultiList::getAt(int index, char &out) const
{
    Node * current = head;
    if (index < 0)
        return false;
    else
    {
```

```
        while (current->next != NULL && current ->index != index)
            current = current ->next;
        if (current != NULL)
        {
            out = current ->data;  //out is set to the character to which index is selected
            return true;
        }
        return false;
    }
}


//prints out the content of the list in order by position, see below
void IndexedMultiList::print() const
{
    Node * current = head;
    while (current != NULL)
    {
        cout << current ->count << " " << current ->data <<  endl;
        current = current ->next;
    }
}



//MUTATORS
/*insert count number of val into the list at position index
returns true iff val was able to fit in list and index was valid and count >=1
valid indices range from 0 up to unique size (inclusive).  So for an empty list, 0 is the only valid index.
insert should do nothing if index is invalid
If the list has 3 A's in the first position, then 0 and 1 are the only valid indices.
*/

bool IndexedMultiList::insert(int index, char val, int count)
{
    //CASE 1: empty
    //CASE 2: beginning
    //CASE 3: middle
    //CASE 4: end

    Node * current = head;
    Node * trailer = current;
    Node * newGuy = new Node (val, count,index,NULL);

    if (index < 0) // Invalid index number
        return false;
    if (head == NULL) // CASE 1: If the list is empty, put the first node at head
    {
        head = newGuy;
        if(index != 0)
        {
            cout<<" warning index " << index << " doesnt exist inserting at index 0"<<endl;
            head->index=0;
        }
        return true;
    }
    else if (index == 0) //CASE 2: Insert item in the beginning if its index is 0.
    {
        newGuy -> next = head;
        head = newGuy;

        current = head ->next; //Start current after head to start shifting everyones index by one.
```

```
        while(current ->next != NULL)
        {
            current ->index+=1; //Shift everyone index after head by 1.
            current = current -> next;
        }
        return true;
    }
    else
    {
        while (current ->next != NULL && newGuy ->index > current ->index)  //Traverses
    through the list for a spot
        {
            trailer = current;  //trailer goes to current's spot
            current = current ->next;   //move current to the next spot
        }
        newGuy-> next = trailer-> next;
        trailer-> next= newGuy;

        current=head; //Current will start at head to start shifting indexes up by 1
        for (int i=0; current; i++) // Looping through to shift everyones index by 1
        {
            current -> index = i;
            current = current -> next;
        }
        return true;
    }

}


/*add amount to item in position index
returns true iff index was valid
change should do nothing if index is invalid
amount could be negative, if adding amount to the current count results in a count of <=
    0 then position should be removed
For example, if there are 3 A's at index 0 and 2 B's at index 1 in the list iml then
iml.change(0, -4)
would remove all the A's and the 2 B's would be at index 0.
*/
bool IndexedMultiList::change(int index, int amount)
{
    Node * current = head;
    while (current != NULL && current ->index != index)
        current = current -> next;
    if (index < 0)
        return false;
    else if (amount + current ->count >= 0)
        current ->count+=amount;
    return true;
}


//finds val in list and adds amount (which could be negative)
//return true iff found, works similarly to change but is based on the
//character to find instead of the position.  If val is in multiple
//locations within the list, then only change the first location
bool IndexedMultiList::update(char val, int amount)
{
    int locate = find(val);
    change(locate, amount);
    return true;
}


//removes 1 instance of val from list,
//returns true iff val was in list, if val is in multiple locations,
```

```
//then only remove from the first location
bool IndexedMultiList::remove(char val)
{
    Node * current = head;
    while (current != NULL && current -> data != val)
        current = current -> next;
    if (current -> data != val)
        return false;
    else
    {
        current->count-=1;
        if (current ->count <= 0)
            delete current;
        return true;
    }
}


//remove position at index, returns count
//of items removed or -1 if index not in use
int IndexedMultiList::remove(int index)
{
    Node * current = head;
    Node * trailer = current;

    if (index < 0 || head == NULL) //If user enter a invalid index number, step out
        return -1;
    else
    {
        while (current != NULL && current -> index != index) //searches through the list
        {
            trailer = current;
            current = current -> next;
        }
        if(current == NULL) ////If the value is not in the list when current reaches NULL↙
    , bail out
            return -1;
        else if (current == head) //CASE 2: Removing the beginning value
        {
            head = head ->next;
            trailer = trailer ->next;
            delete current;

            while (trailer != NULL) //Prepare trailer to shift everyones index down by 1.
            {
                trailer -> index-=1; //Shift everyones index down by 1.
                trailer = trailer ->next;
            }
        }
        else
        {
            trailer ->next = current ->next; //CASE 3: Removing middle and end
            delete current;

            trailer = trailer ->next;
            while (trailer != NULL) //Prepare trailer to shift everyones index down by 1.
            {
                trailer->index-=1;  //Shift everyones index down by 1.
                trailer = trailer ->next;
            }
        }
        return -1;
    }
}
```

```
//removes all instances of val from list
//returns number of items removed, which could be 0, if val is in
//multiple locations it should remove only the first location
int IndexedMultiList::removeAll(char val)
{
    Node * current = head;
    Node * trailer = current;

    if (head == NULL) //If user enter a invalid index number, step out
        return -1;
    else
    {
        while (current != NULL && current -> data != val) //searches through the list
        {
            trailer = current;
            current = current -> next;
        }

        if(current == NULL) //If the value is not in the list when current reaches NULL, bail out
            return -1;

        if (current == head) //CASE 2: Removing the beginning value
        {
            head = head ->next;
            trailer = trailer ->next;
            delete current;

            while (trailer != NULL) //Prepare trailer to shift everyones index down by 1.
            {
                trailer->index-=1; //Shift everyones index down by 1.
                trailer = trailer ->next;
            }
        }

        else
        {
            trailer ->next = current ->next; //CASE 3: Removing middle and end
            delete current;

            trailer = trailer ->next;
            while (trailer != NULL) //Prepare trailer to shift everyones index down by 1.
            {
                trailer->index-=1;  //Shift everyones index down by 1.
                trailer = trailer ->next;
            }
        }
        return -1;
    }
}
```

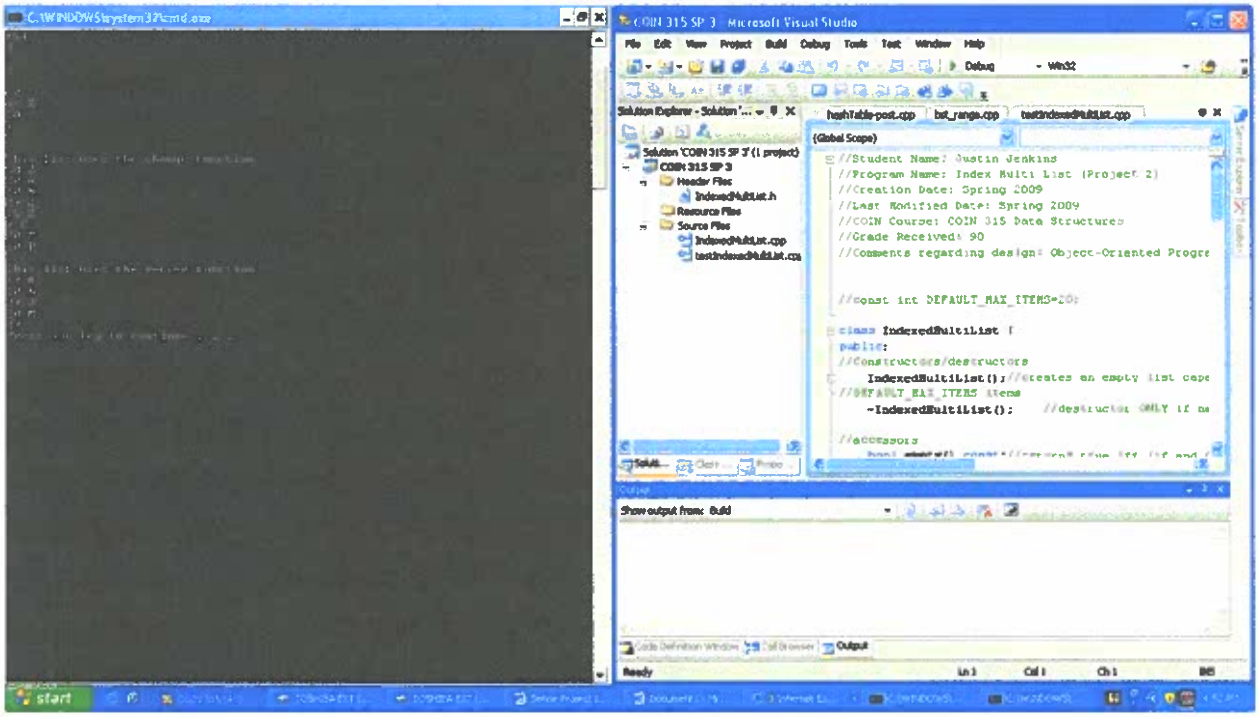
```cpp
#include <iostream>
#include "IndexedMultiList.h"
using namespace std;

void main()
{
    IndexedMultiList iml;

    iml.insert(0,'A',20);   //Inserts 20 A's in index 0
    iml.insert(1, 'B', 5);  //Inserts 5 B's in index 1
    iml.insert(1, 'C', 9);
    iml.insert(1, 'N', 10);
    iml.insert(3, 'M', 20);
    iml.insert(2, 'J', 30);
    iml.insert(0, 'Z', 50);
    iml.insert(4, 'Z', 10);

    iml.print();
    cout << endl;
    cout << iml.uniqueSize()<< endl;
    cout << iml.size() << endl;
    cout << iml.find('J') << endl;   //finds the item J in the list

    cout << endl;
    cout << iml.countItem('Z') <<endl;   //counts how items Z has
    cout << endl;

    cout << iml.totalItemCount('Z') << endl;

    //for getAt function
    char a;
    if (iml.getAt(4,a))   //  if there's an item at index 4
        cout<<"a= "<<a<<endl;  //prints the item at index 4
    else
        cout <<"Index not found." <<endl;
    //for getAt function

    cout << iml.countPosition(4) << endl;
    cout << iml.change(4, 5) << endl;  //would remove 5 items at index 4
    cout << iml.update('B', 5) << endl;  // add 5 items to letter B
    iml.remove('J');
    cout << endl;

    cout << "This list uses the change function"<< endl;
    iml.print();

    cout << endl;
    cout << "This list uses the remove function" << endl;
    iml.remove(7);   //removes an item at index 7
    iml.removeAll('C');  //removes all items of C
    iml.removeAll('Z');
    iml.removeAll('Z');
    iml.remove('A');

    iml.print();
    cout <<iml.countPosition(2) << endl;
}
```

```
//Student Name: Justin Jenkins
//Program Name: Binary Search Tree
//Creation Date: Spring 2009
//Last Modified Date: Spring 2009
//COIN Course: COIN 315 Data Structures
//Grade Received: 90
//Comments regarding design: Object-Oriented Programming type program.  Written in C++.

#include <iostream>
using namespace std;

template <typename itemtype>
class BST {//unbalanced binary search tree, no duplicates allowed, templated
public:
  BST() {root=0;}//empty tree

  ~BST() {  clear(root);}

  bool empty() const {return (root==0);} //return true iff no nodes

  bool insert (itemtype val) { //insert val into a correct spot, return false if
    duplicate or out of memory
    if (empty()) {
      root=new Node(val);
      return (root != 0); //return true if root is not NULL, NULL if no memory
    }
    return insert(val, root);
  }

  bool search (itemtype val) const{ //return true iff val found in tree
    return search(val, root);
  }

  void traverseInOrderPrint() const {  // to print tree in order
    traverseInOrderPrint(root);
    cout<<endl;
  }

  int printRange(int start, int end) const {//this should print all values in the tree
    that are
    //>= start and <= end (so it is inclusive)
    //print a single space between each item
    int temp= printRange(start, end, root);
    cout<<endl;
    return temp;
  }

private:
  struct Node {//constructor provided for simplicity of insert
    Node (itemtype d) : data(d), lChild(0), rChild(0) {}
    itemtype data;
    Node *lChild, *rChild;
  };
  Node * root;

  //below are recursive helpers for public methods with the same name
//version w/o using pass by reference, this is why we didn't do it this way
  bool insert (itemtype val, Node *cur) {
    if (cur->data==val) return false;
    if (val < cur->data) {//go left
      if (cur->lChild)    return insert(val, cur->lChild);//no left so insert here
      cur->lChild=new Node(val);
      return (cur->lChild != 0);
    }
    else {
      if (cur->rChild)    return insert(val, cur->rChild);//no right so insert here
```

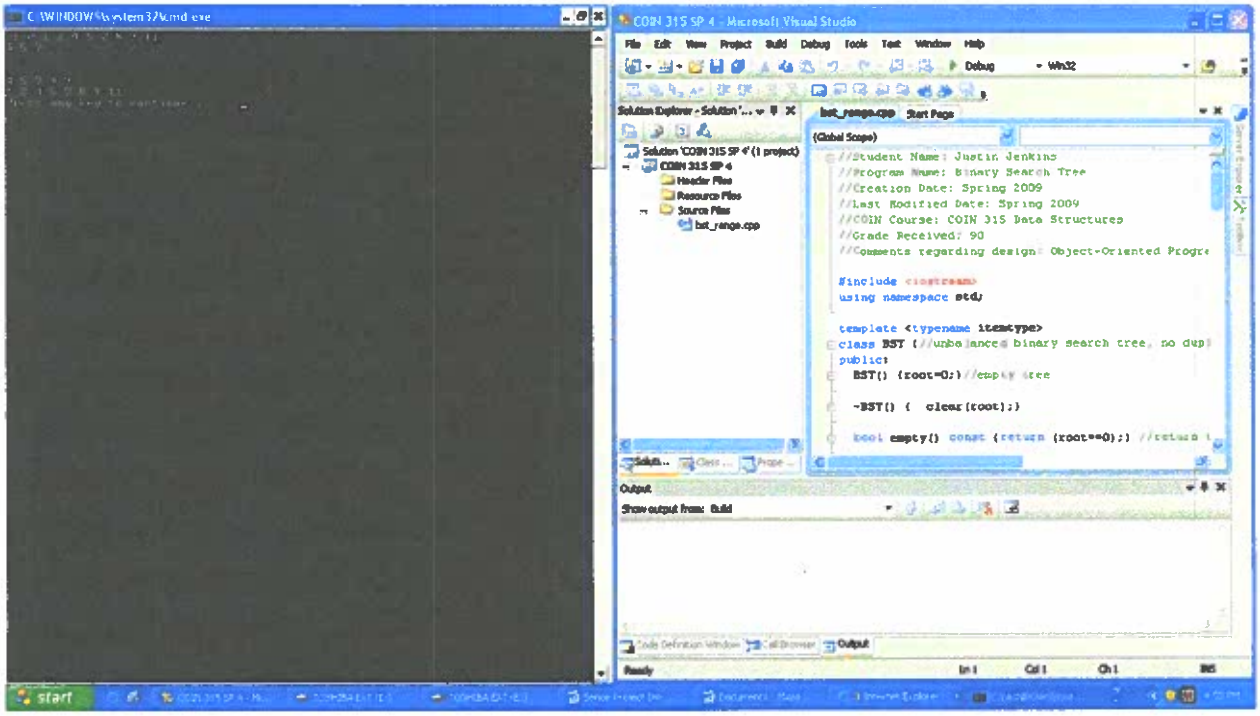
```
      cur->rChild=new Node(val);
      return (cur->rChild != 0);
    }
  }
  void traverseInOrderPrint(Node * cur) const {//useful for debug
    if (cur==0) return;
    traverseInOrderPrint(cur->lChild);  //recursively call this function
    cout <<cur->data<<" ";
    traverseInOrderPrint(cur->rChild);  //recursively call this function
  }
  void clear(Node * cur) {//post order traversal, delete for visit
    if (cur == 0) return;
    clear (cur->lChild);
    clear (cur->rChild);
    delete cur;
  }

  bool search (itemtype val, Node *cur) const {
    if (cur==0)           return false;
    if (cur->data == val) return false;
    if (val < cur->data)  return search(val, cur->lChild); // val smaller so we go left
    else                  return search(val, cur->rChild); // val is bigger so we go
    right
  }

  int printRange(int start, int end, Node * cur) const {
    //need to implement this method
        if (cur == NULL)
            return -1;
        printRange(start, end, cur->lChild);  //recursively call this function
        if (cur->data >= start && cur ->data <= end)
            cout <<cur->data<<" ";
        printRange(start, end, cur->rChild);  //recursively call this function
  }

};

void main() {//some test code
  BST<int> tree;
  int arr[10] = {5,9,11,0,2,1,4,8,-1,7};
  for (int i=0; i<10; i++) tree.insert(arr[i]);
  tree.traverseInOrderPrint();
  tree.printRange(3,7);
  tree.printRange(15,20);  //prints when both values that are not in the tree
  tree.printRange(-10, -8);  //prints when both values that are not in the tree
  tree.printRange(4, 10);  //prints when the 10 is not in the tree
  tree.printRange(1, 11);  //prints when both values are in the tree
  tree.search(6);
}
```

COIN 315 Data Structures

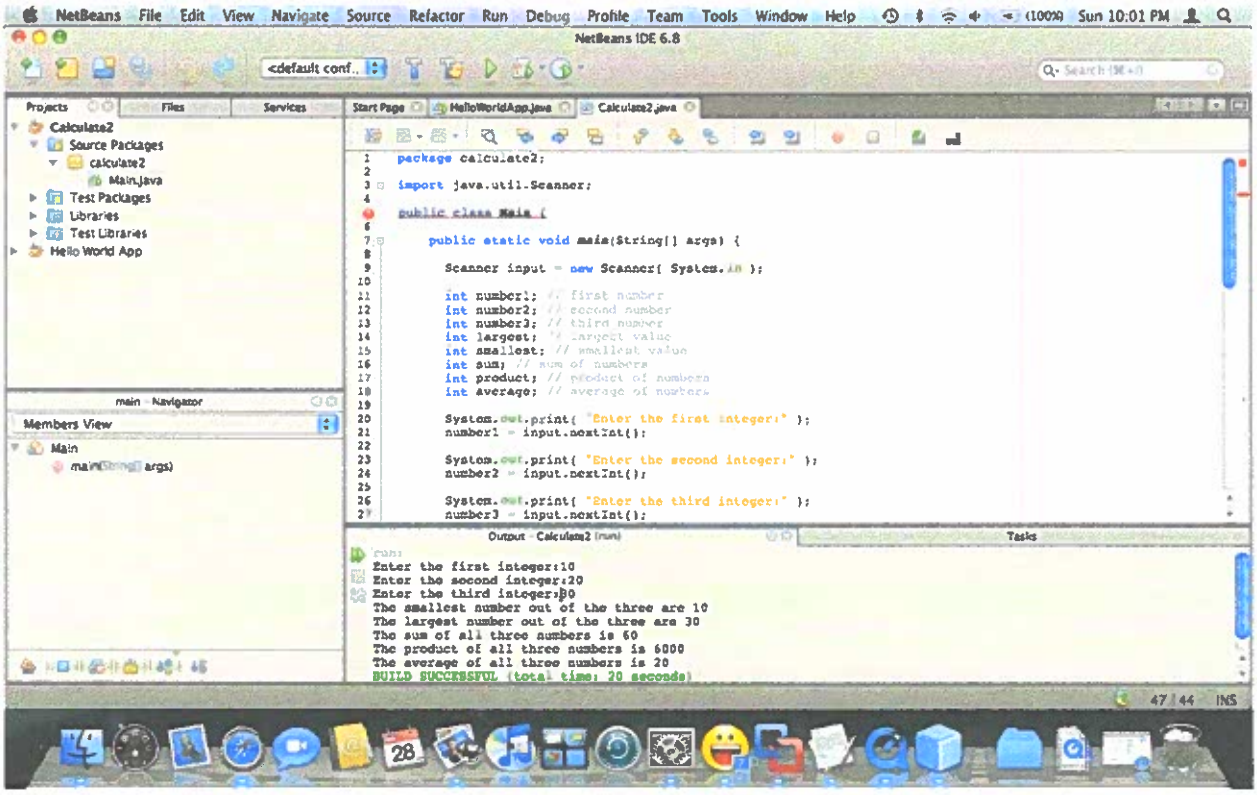
Linked List (Indexed Multi-List Type)



Binary Search Tree

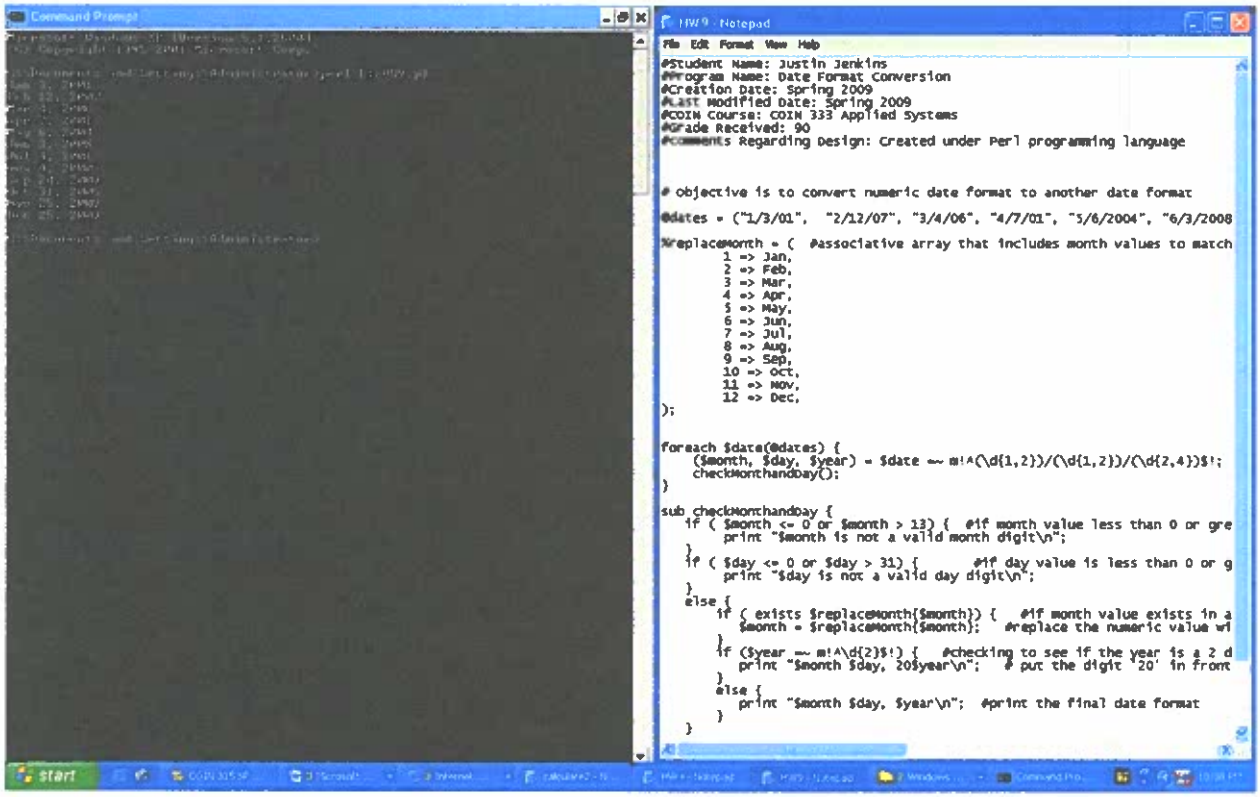


```
#Student Name: Justin Jenkins
#Program Name: Number Calculations
#Creation Date: Spring 2008
#Last Modified Date: Spring 2008
#COIN Course: COIN 325 Java I
#Grade Received: 90
#comments Regarding Design: Written in Java programming language

package calculate2;

import java.util.Scanner;

public class calculate2{

    public static void main(String[] args) {

      Scanner input = new Scanner( System.in );

      int number1; // first number
      int number2; // second number
      int number3; // third number
      int largest; // largest value
      int smallest; // smallest value
      int sum; // sum of numbers
      int product; // product of numbers
      int average; // average of numbers

      System.out.print( "Enter the first integer:" );
      number1 = input.nextInt();

      System.out.print( "Enter the second integer:" );
      number2 = input.nextInt();

      System.out.print( "Enter the third integer:" );
      number3 = input.nextInt();

      largest = number1; // assume number1 is the largest
      smallest = number1; // assume number1 is the smallest

      if( number2 > largest)
        largest = number2;
      if( number3 > largest)
        largest = number3;
      if( number2 < smallest)
        smallest = number2;
      if( number3 < smallest)
        smallest = number3;

     System.out.printf( "The smallest number out of the three are %d\n", smallest);
     System.out.printf( "The largest number out of the three are %d\n", largest);

     // perform calculations

     sum = number1 + number2 + number3;
     product = number1 * number2 * number3;
     average = sum / 3;

     System.out.printf( "The sum of all three numbers is %d\n", sum);

     System.out.printf( "The product of all three numbers is %d\n", product);

     System.out.printf( "The average of all three numbers is %d\n", average);
    }
```


NetBeans File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
NetBeans IDE 6.8
Projects
Files
Services
Calculate2
Source Packages
calculate2
Main.java
Test Packages
Libraries
Test Libraries
Hello World App
Start Page
HelloWorldApp.java
Calculate2.java
package calculate2;
import java.util.Scanner;
public class Main {
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
int number1; // first number
int number2; // second number
int number3; // third number
int largest; // largest value
int smallest; // smallest value
int sum; // sum of numbers
int product; // product of numbers
int average; // average of numbers
System.out.print("Enter the first integer:");
number1 = input.nextInt();
System.out.print("Enter the second integer:");
number2 = input.nextInt();
System.out.print("Enter the third integer:");
number3 = input.nextInt();
main - Navigator
Members View
Main
main(String[] args)
Output - Calculate2 (run)
Tasks
run:
Enter the first integer:10
Enter the second integer:20
Enter the third integer:30
The smallest number out of the three are 10
The largest number out of the three are 30
The sum of all three numbers is 60
The product of all three numbers is 6000
The average of all three numbers is 20
BUILD SUCCESSFUL (total time: 20 seconds)
47 | 44
INS

```
#Student Name: Justin Jenkins
#Program Name: Date Format Conversion
#Creation Date: Spring 2009
#Last Modified Date: Spring 2009
#COIN Course: COIN 333 Applied Systems
#Grade Received: 90
#comments Regarding Design: Created under Perl programming language


# Objective is to convert numeric date format to another date format

@dates = ("1/3/01",  "2/12/07", "3/4/06", "4/7/01", "5/6/2004", "6/3/2008",
"7/4/01", "8/4/00", "9/20/09", "10/31/05", "11/25/09", "12/25/2009");  #numeric
dates to convert

%replaceMonth = (  #associative array that includes month values to match the string
format of the month
        1 => Jan,
        2 => Feb,
        3 => Mar,
        4 => Apr,
        5 => May,
        6 => Jun,
        7 => Jul,
        8 => Aug,
        9 => Sep,
        10 => Oct,
        11 => Nov,
        12 => Dec,
);


foreach $date(@dates) {
    ($month, $day, $year) = $date =~ m!^(\d{1,2})/(\d{1,2})/(\d{2,4})$!; #(pattern
matching) month and day values can have 2 digits
    checkMonthandDay();
#year value can have 2 to 4 digits
}

sub checkMonthandDay {
   if ( $month <= 0 or $month > 13) {  #if month value less than 0 or greater than
13
        print "$month is not a valid month digit\n";
   }
   if ( $day <= 0 or $day > 31) {       #if day value is less than 0 or greater than
31
        print "$day is not a valid day digit\n";
   }
   else {
       if ( exists $replaceMonth{$month}) {   #if month value exists in associate
array "replacemonth",
          $month = $replaceMonth{$month};   #replace the numeric value with the
string of the month's name
       }
       if ($year =~ m!^\d{2}$!) {   #checking to see if the year is a 2 digit.
          print "$month $day, 20$year\n";   # put the digit '20' in front of the
existing 2 digit year
       }
       else {
          print "$month $day, $year\n";  #print the final date format
       }
   }
```

COIN 333 Applied Systems



COIN 332 Applied Networking Paper

Grade Received: 100

Justin Jenkins

Applied Networking

January 25, 2010

Dr. Lin

Media that are available today for consumers are moving digital. Music is included in this category which users can download music from the internet without going to a store. The problem the music industry is having is users downloading music without paying the right to keep the material. Programs like LimeWire, Ares, and uTorrent are peer to peer programs (P2P) that allow users to download multimedia content without paying for it. Downloading music illegally is considered not to be ethic but due to the easy access of obtaining copyright music, most users ignore the warning signs of breaking the law.

Downloading illegal music wasn't a major issue in the 1990's until 1999 and the millennium. The traditional way of obtaining music was from a music store where they provided cassette tapes and compact discs (CD) for sale. In 1999, an online music sharing service named Napster was created to allow users download music for free. The music industry noticed Napster's activity after reporting a 10 percent fall in sales of2001[1]. This prompted the Recording Industry Association of America (RIAA) to file a lawsuit against Napster for unauthorized use of audio copyright material. In July 2001, the RIAA forced Napster to "shut down its entire network in order to comply with the injunction[2]". The injunction simply meant to "prevent Napster from trading copyright music over its network[2]".

Some people have mixed opinions about downloading music. Depending on the age group, people who are younger than 30 years old represent the majority of users on the P2P network. According to a *USA WEEKEND* magazine poll that interviewed teenagers, "19% frequently download music, 26% occasionally download music and 55% never or rarely download music[3]". The study also showed that, "54% see nothing wrong[3]" with downloading music from the Internet. An estimated 10%

say "it cheats the artists and shouldn't be done, while 15% agree it cheats the artists, but I still think its OK[3]". On the other hand, TechCrunch's website reports that "people who are older than 30 think that downloading music is ethically wrong[3]". These results may have been influenced from teens having more knowledge with computers than adults. You can also consider that a music store was the most convenient way for adults to purchase music since the internet did not provide access to songs and albums yet.

So is downloading music ethically right? In some situations it may be okay. Some users may want to download music to backup their original songs. Others may receive music from their friends by email or social networking websites. As long as the song has been purchased, there should not be an issue but there are other situations where people will take advantage of getting music and not supporting their artists financially. As previously mentioned earlier from the *USA Magazine* poll, the majority of users do not see this issue as a big deal assuming artists will still receive their money one way or another. Due to the number of P2P networks that are available today, the music industry has found that it is difficult to stop the music sharing services. Other alternatives the music industry and the RIAA seeking are actions "to work with Internet Service Providers (ISP) who will use a three strike warning system for file sharing, and upon the third strike will cut off internet service all together[4]". This may be great deal for ISPs to get money from the music industry but it may hurt its customers with restrictions.

1. Azoz. "RIAA Statistics Don't Add Up to Piracy". Available from http://www.azoz.com/music/features/0008.html. Internet. accessed on 25 January 2010
2. Wikipedia. "Napster". Available from http://en.wikipedia.org/wiki/Napster . Internet. Accessed on 25 January 2010
3. TechCrunch. "Stealing Music: Is It Wrong or Isn't It?" available from http://www.techcrunch.com/2009/03/31/stealing-music-is-it-wrong-or-isnt-it/. Internet. Accessed on 25 January 2010.
4. Wikipedia. "Recording Industry Association of America"; available from http://en.wikipedia.org/wiki/Recording_Industry_Association_of_America. Internet. Accessed on 25 January 2010.

COIN 431 Advanced Operating Systems Ethics Paper

Grade Received: 92

Video Game Violence

Justin Jenkins
Professor Gary Underwood
Advanced Operating Systems
September 10 2009

92

In these hard times, it's hard to find anything positive from the economy. But while everyone is going down, video games are going up. Business watch company, Pricewaterhouse Coopers, released a report "[estimating] that the video game market will increase from $31.6 billion in 2006 to $48.9 billion in 2011" (Scanlon). What make video games popular is of course its games but particularly the mature games are rushing the entertainment industry towards the top. According to Gamesindustry.biz, they quoted that "A new study of US game sales has highlighted that games rated M for Mature have the highest average gross sales in the region" (Martin). That may alert some parents who have children to play video games. The commotion that is coming from violent video games is that it poses a negative effect towards young consumers who play these games. To ease parent's worries, there are laws that protect young children from buying violent video games. Even from an Christian viewpoint, children that are exposed to negative activity will affect their lives down the road. By enforcing these laws and spreading the Christian view, these video games will not pose a threat to society.

Before video games can be released to retailers, they are rated by a rating board system from the Entertainment Software Rating Board (ESRB). The "ESRB assigns computer and video game content ratings, enforces industry-adopted advertising guidelines and helps ensure responsible online privacy practices for the interactive entertainment software industry" (ESRB). The company rates video game content that includes "typical gameplay, missions, and cutscenes, along with the most extreme instances of content across all relevant categories including but not limited to violence, language, sex, controlled substances and gambling" (ESRB). Each game gets an overall rating of either EC for Early Childhood, E for Everyone, E 10+ for Everyone 10 and up, T for Teen, M for Mature or AO for Adults only. If the game poses enough violence that may not be suitable for young children, then the game gets a mature rating (rated M). ESRB

quotes that "Titles in this category may contain intense violence, blood and gore, sexual content and/or strong language" (ESRB). This helps parents to be alert and identify what game is acceptable to children.

Although ESRB rates every video game that is released, they do not enforce who can buy the game. Each individual store has their own restrictions to sell mature games to minors. There were bills introduced that would add more enforcement to violent video games but was unsuccessful. The Family Entertainment Protection Act, introduced by Senator Hillary Clinton and co-sponsored by Joseph Lieberman, was a "bill to limit the exposure of children to violent video games" (Govtrack). Another bill that was introduced but not yet signed into law was the Video Game Rating Enforcement Act. This act is "a bill to prohibit the distribution or sale of video games that do not have age-based content rating labels, to prohibit the sale or rental of video games with adult content ratings to minors, and for other purposes" (Open Congress). These bills would be great if they were taken into law and warn retailers to not sell inappropriate games to minors but overall, retailers are taken this issue seriously to satisfy their older primary consumers.

In a Christian view, violent video games can brainwash minors in becoming violent later in life. In a survey by the Kaiser Family Foundation, quoted "significant correlations between routine play of M-rated games and greater self-reported involvement in physical fights (Olsen)" and that "aggressive or hostile youths may be drawn to violent games" (Olsen). No matter if the violence is from entertainment or another subject, minors who are exposed to negative activity will affect their lives later on. The reason behind this issue is that children who are between the ages 5 and 17 are in a developing stage of learning moral values. In this timeline, their minds do not understand that violence in video games should not be enacted in real life. It's the same

process of learning right from wrong. On the hand minors must know the consequences of violence instead of being exposed to it. In this way, they will have a better knowledge of understanding what violence is.

If more churches were involved with minors about understanding violence in video games, the issue would be less of a problem. Most churches use youth programs to allow minors to communicate with others and work with each other in a Christian-friendly environment. A typical program teaches minors the values of becoming a better Christian by getting involved in their church and reading the Bible. The program also may offer family video games that the children may like. The program is a great way to teach minors about violence in video games and what alternatives are best for them. To set an example from the Bible, in Philippians 4:13, the Bible quotes "I Can Do All Things through Christ Who Strengthens Me". Minors who participate in their church and learn Christian values are most likely to stay out of trouble. The reason why is that they are not exposed to much violence as they are using their time to learn good moral values. This also makes them a better person in life by making better decisions to what is good for them.

Although there are retailers that enforce minors from buying violent games, understanding the violence through Christianity is the most effective way. The overall enforcement from the parents, retailers and the help of churches, will keep violence from video games a small problem.

Work Cited

"About ESRB". Electronic Software Rating Board. 9 September 2009. http://www.esrb.org/about/index.jsp

"Game Rating and Descriptor Guide". Electronic Software Rating Board. 9 September 2009. http://www.esrb.org/ratings/ratings_guide.jsp

Martin. Matt. "Study Shows Mature-rated Games Selling Best". Gamesindustry.biz. http://www.gamesindustry.biz/articles/study-shows-mature-rated-games-selling-best. 9 November 2007. 9 September 2009.

Olsen, Cheryl. "Children and Video Games: How Much Do We Know". Psychiatric Times. http://www.psychiatrictimes.com/display/article/10168/54191?verify=0. 1 October 2007. 9 September 2009.

Scanlon, Jessie. "The Video Game Industry Outlook: $31.6 Billion and Growing". Business Week. http://www.businessweek.com/innovate/content/aug2007/id20070813_120384.htm. 13 August 2007. 9 September 2009.

"Rating Process". Electronic Software Rating Board. 9 September 2009. http://www.esrb.org/ratings/ratings_process.jsp

"S.3315 - Video Game Rating Enforcement Act". Open Congress. 9 September 2009. http://www.opencongress.org/bill/110-s3315/show.

S. 216 Family Entertainment Protection Act". Govtrack.us: A Civic Project to Track Congress. http://www.govtrack.us/congress/bill.xpd?bill=s109-2126.

COIN 315 Data Structures Ethics Paper

Grade Received: 87

Net Neutrality

Justin Jenkins
Data Structures
September 10, 2008

Net Neutrality

Our daily routine throughout the day is most likely dealt with technology including the Internet. One term that the average Internet user may not know about but is obviously a part of it is what we call net neutrality. The meaning of net neutrality "prevents Internet providers from blocking, speeding up or slowing down Web content based on its source, ownership or destination" (Google). This means that every website shares the same speed whether it's a corporation or a personal website. Internet users also have equal rights of net neutrality which means that "content on the Internet should be equally accessible by all users" (Bosworth). Web-based companies including Google and eBay are in favor for net neutrality while Internet service providers, Comcast and Verizon are against this issue. Internet Service Providers has a disagreement against neutralization by providing more bandwidth to certain websites depending on their quality service over the internet. ISPs also limit users to a certain bandwidth due to excess of downloading data to certain websites. This may be great for ISP's manage their data more efficiently, but they are also creating a bad image among their users by downgrading their service. The decision to go against net neutrality is not only wrong but also discriminates people's rights to use the Internet as a source of opportunity, business, and entertainment.

As the Internet soars at an amazing rapid rate, the decision to demolish neutralization on the Internet is disappointing. It's now becoming a legal issue whether it is right for ISP companies to control the way people use the Internet. I believe it is right for the Internet to continue to be neutralized because it bring more business into cyberspace where people are now relying on the Internet to shop, do research and find innovate ways to communicate to other users around the world. The conflict of ISPs abolishing net neutrality is all about money when

Internet.." (Bosworth) and will make sure "that no consumer is going to be blocked" (Bosworth). It's an unusual scenario we are seeing since we know the government has similar ways with companies like the RIAA for cracking down illegal action. The RIAA sued networks like LimeWire for "encouraging sharing" (Mennecke) to protect their artists from losing revenue but the lawsuit wasn't on a level where net neutrality rights kicked in. The government believes that altering people use of Internet is like taking away their rights. Net Neutrality is known to be the "First Amendment of the Internet" (Net Neutrality) where people can use Internet with freedom but without interference. The government will stop illegal actions but will not violate people rights from using the Internet as a source for information. This is one reason why net neutrality protects Internet users to access file-sharing networks.

There are some logical reasons to why net neutrality is important for cyberspace. Internet users pay ISPs to access information on the Internet and by using that feature, they must pay their service provider to continue using the Internet. If a person has to pay their hard-earned cash to keep their service running, then they should have every right to access any information they like. ISPs that block their users from certain websites are only losing subscribers to their great services. Websites including ESPN, CNN, eBay, Amazon, Playboy, etc. are places where users enjoy using the Internet. Blocking these websites or limiting their bandwidth for access will only hurt the economy when most consumers rely on the Internet to invest in products and media. Comcast reason of blocking its users from BitTorrent because of "reasonable network management" (Bosworth) is just a bad excuse. ISPs need to invest more money for higher bandwidth data than making the situation worse. If it is helping the economy and creating more business to users then it shouldn't be changed and I am sure the government doesn't have a problem with it.

Work Cited

not MLA

1. Google Help Center. 2008.

   <http://www.google.com/help/netneutrality.html>

2. Comcast Blocks Subscribers From Some Services. Ed. Martin H. Bosworth. 2007. 21 Oct, 2007

   http://www.consumeraffairs.com/news04/2007/10/comcast_blocking.html

3. Save The Internet

   <http://www.savetheinternet.com/=faq>

4. Abandoning net neutrality discourages improvements in service. Ed. Cathy Keen. 2007. 7

   Mar, 2007 http://news.ufl.edu/2007/03/07/net-neutrality/

5. Comcast Blocks Subscribers From Some Services. Ed.

   http://www.consumeraffairs.com/news04/2007/10/comcast_blocking.html

6. FCC To investigate Comcast For Blocking Net Traffic. Ed. Martin H. Bosworth. 2008. 9 Jan.

   2008 http://www.consumeraffairs.com/news04/2008/01/comcast_blocking_fcc.html

7. LimeWire Sued by The RIAA. Ed. Thomas Mennecke. 2006. 4 Aug 2006

   http://www.slyck.com/news.php?story=1258

8. Net Neutrality. 2008. 18 Apr 2008

http://searchnetworking.techtarget.com/sDefinition/0,,sid7_gci1207194,00.html#/

9. Comcast, Cox Caught Blocking BitTorrent. Ed. Martin H. Bosworth. 2008 16 May 2008

http://www.consumeraffairs.com/news04/2008/05/comcast_cox.html

COIN 431 Advanced Operating Systems Presentation

INTERNET EXPLORER 9

Justin Jenkins

Moving Beyond Version 8

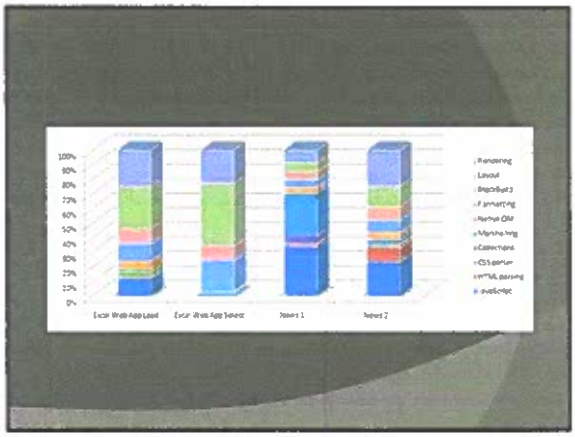
- Why is IE 9 mentioned so early?
  - To give users a different view on what's to come
  - Provide feedback on what needs to change
  - A chance to regain the lead in the browser wars.

Looking Forward

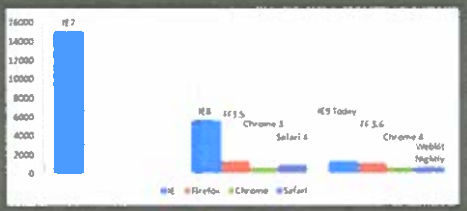
- What changes can we see for IE?
  - Faster installation
  - Enable and Disable browser plug-ins
  - Twitter Client Integration
    - Improving communication between users within IE
  - Smoother importing and organizing favorites
  - Better integration to Microsoft Live SkyDrive
    - A Cloud storage service that stores files over the internet
  - Hardware Acceleration (Major Feature)

Hardware Acceleration

- IE will be the first browser to use hardware acceleration to handle webpages.
  - Current browsers rely on the CPU to do the work
  - No other internet browser have this feature



Script Performance



The chart above shows the relative performance of different browsers on the same machine running the SunSpider (Apple's Webkit) test

Hardware Acceleration (Cont.)

- IE will use DirectX family of Windows APIs to enable many advances for web developers
- Not only graphics but fonts will also take advantage of DirectX technology in IE. (Next Slide)

Font Example

- *96 point Gabriola on a Lenovo X61 ThinkPad at 100% Zoom using GDI (note jaggies)*

- *96 point Gabriola on a Lenovo X61 ThinkPad at 100% Zoom Direct2D (without jaggies)*

Benefits

- With hardware acceleration this feature will allow:
  - The CPU to work less with the help of a graphics card
    - Improves overall performance from the computer
  - Multimedia websites to perform better. (Websites containing 3D graphics, flash video, audio, etc.)
  - IE to hopefully regain the #1 spot in the browser wars.

Sources

- http://blogs.msdn.com/ie/archive/2009/11/18/an-early-look-at-ie9-for-developers.aspx

COIN 431 Advanced Operating Systems Presentation

FREE SPACE ALLOCATION

Justin Jenkins
Operating Systems

Linux File system

- Ext (Extended File System) is the file system used for Linux
- Ext4 is the latest native file system for Linux
  - 4th version of the extended file system released for Linux
  - Released in December 2008
  - Improved Performance
  - Supports a max HD volume up to 1 Exabyte (1 million GB) and files sizes up to 1 tebibytes (1,024 GB)

Ext4 Allocation Management

- Delayed Allocation
  - Also known as "allocate-on-flush"
  - Delays block allocation until data is written on disk
    - Deducts value from free space counter but delays obtaining the actual free space
  - Data is added to memory until it is flushed to storage
  - Improves performance and reduces fragmentation
- Disadvantage to Delayed Allocation
  - Data Loss can occur on power outage

Ext4 Allocation Management

- Multi block Allocation
  - Allocates multiple blocks on the same call oppose to a single block per call (ext3 problem).
  - Useful with delayed allocation
  - Doesn't allocate unnecessary free space

How Can We Free Up Space?

- Introducing ftruncate() function
  - Basically decreases the file size to a given length

```
#include <unistd.h>   int ftruncate(int fildes, off_t length);
```

  - intfildes – refers to a regular file
  - Off_t length – refers to a value of the file size should truncate to

Ftruncate() rules

```
#include <unistd.h>   int ftruncate(int fildes, off_t length);
```

- If intfildes is something other than a file, the function fails
- If off_t length value is less than the file size, the file size decreases to the specified length.
- If off_t length value is greater than the file size, the file size increases